

Sling Model & OSGi

Annotations — Complete Reference Guide

This reference guide covers every major annotation used in **Adobe Experience Manager (AEM)** development — from Sling Model injection annotations to OSGi Declarative Services. Each annotation includes a detailed explanation, full code example, expected output, and guidance on when to use it.

Sling Model Annotations	OSGi Annotations	Key Differences
@Model	@Component	@Inject vs @ValueMapValue
@ValueMapValue	@Reference	@Named vs name= attribute
@ChildResource	@Activate / @Deactivate / @Modified	@Optional vs @Required vs @Default
@OSGiService	@ObjectClassDefinition + @AttributeDefinition	@OSGiService vs @Reference
@Self, @Via	@Designate	@ChildResource vs @Self vs @Via
@PostConstruct	EventHandler pattern	Sling vs OSGi lifecycle
@SlingObject, @ScriptVariable	Scheduler pattern	
@Exporter (JSON)		

1. Sling Model Annotations

Sling Models provide a clean, annotation-driven way to map JCR content into Java objects. These annotations live in `org.apache.sling.models.annotations` and `org.apache.sling.models.annotations.injectorspecific`.

@Model

The **entry-point annotation**. Every Sling Model class must have this. It tells Sling what this class adapts from (Resource or Request), what interfaces it implements, and the default injection strategy.

```
@Model(
    adaptables = {Resource.class, SlingHttpServletRequest.class},
    adapters = {MyModel.class, IMyModel.class},
    resourceType = "myapp/components/mycomp",
    defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL
)
public class MyModel implements IMyModel {
    // fields injected here
}
```

Attribute	Type	Description	Required?
adaptables	Class[]	Resource.class and/or SlingHttpServletRequest.class	Yes
adapters	Class[]	Interface(s) this model adapts to — used by adaptTo()	Recommended
resourceType	String[]	JCR sling:resourceType — required for .model.json exporter	For exporter
defaultInjectionStrategy	Enum	OPTIONAL (null on missing) or REQUIRED (throw on missing)	No

✓ **Rule:** Always list the interface in adapters=, not the impl class. Use OPTIONAL strategy in production — it prevents a misconfigured component from breaking an entire page.

@ValueMapValue

Reads a value directly from the resource's **ValueMap (JCR properties)**. The most important and most-used Sling Model annotation. Supports automatic type coercion — String→int, String→boolean, String[]→List, etc.

Code

Output / Result

```
// Field name matches JCR property name
@ValueMapValue
private String title;
// Override name for colon-separated JCR keys
@ValueMapValue(name = "jcr:title")
private String jcrTitle;
// Type coercion – JCR stores "149" as String
@ValueMapValue
private int price;
// Multi-value String[] auto → List<String>
@ValueMapValue
private List<String> tags;
// Boolean coercion from "true"/"false"
@ValueMapValue
private boolean hideInNav;
// Default fallback when property missing
@ValueMapValue
@Default(values = "Click here")
private String buttonLabel;
```

JCR: title = "Hello AEM"

title → "Hello AEM"

JCR: jcr:title = "My Page"

jcrTitle → "My Page"

JCR: price = "149" (String)

price → 149 (auto-coerced)

JCR: tags = ["sport", "news"]
(String[])

tags → ["sport", "news"]
(List<String>)

JCR: hideInNav = "true" (String)

hideInNav → true (auto-coerced)

JCR: buttonLabel = (NOT SET)

"Click here" ← from @Default

★ **Key rule:** Use `@ValueMapValue` for ALL JCR property reads — it is explicit, fast (direct ValueMap lookup), and handles type coercion. Never use `@Inject` when you know the value comes from JCR.

@ChildResource

Injects a **child JCR node** relative to the current resource. Can inject as `Resource`, `List<Resource>`, or auto-adapt each child to another Sling Model.

Code

```
// JCR tree:
// /jcr:content/carousel
// ■■■ items/
// ■ ■■■ slide0/ → title="Slide 1"
// ■ ■■■ slide1/ → title="Slide 2"
// ■■■ cq:responsive/
// Single child node as Resource
@ChildResource
private Resource items;
// Child with colon in name needs name=
@ChildResource(name = "cq:responsive")
private Resource responsive;
// All children as List<Resource>
@ChildResource
private List<Resource> items;
// Auto-adapt each child to SlideModel
@ChildResource
private List<SlideModel> items;
```

Output / Result

items → Resource at
.../carousel/items

responsive → Resource at
.../cq:responsive

items → [Resource(slide0),
Resource(slide1)]

items → [SlideModel, SlideModel]
items.get(0).getTitle() → "Slide 1"

✓ **Auto-adaptation magic:** When typed as `List<SlideModel>`, Sling calls `resource.adaptTo(SlideModel.class)` for each child automatically — zero boilerplate loops. Always null-check single child resources (OPTIONAL strategy = null when node missing).

@PostConstruct

Marks a method to be called **after all injections are complete** but before the model is returned to the caller. This is the only safe place for logic that depends on injected fields — they are still null inside constructors.

Code

```
@ValueMapValue
private String rawTitle;

@ValueMapValue
private String fileReference;

@SlingObject
private ResourceResolver resolver;
private String cleanTitle;
private String imageName;

@PostConstruct
protected void init() {
    // Safe – all injected fields ready
    cleanTitle = rawTitle != null
        ? rawTitle.trim() : "Default";
    if (fileReference != null) {
        Resource img = resolver
            .getResource(fileReference);
        imageName = img != null
            ? img.getName() : "";
    }
}
```

Output / Result

JCR: rawTitle = " hello world "

JCR: fileReference =
"/content/dam/hero.jpg"

Execution order:

1. Constructor runs (fields = null)
2. @ValueMapValue fields injected
3. @PostConstruct init() fires
4. Model returned to caller

cleanTitle → "hello world" (trimmed)

imageName → "hero.jpg"
(resolved from DAM path)

■ **Critical rule:** NEVER use injected fields inside the constructor — they are null at that point. All initialization logic that depends on injected values must go in `@PostConstruct`. The method name can be anything (init, setup, activate — your choice).

@SlingObject and @ScriptVariable

`@SlingObject` injects Sling framework objects (ResourceResolver, Resource, Request, Response).

`@ScriptVariable` injects AEM HTL script bindings (currentPage, pageManager, designer). Both are cleaner and faster than `@Inject` for these types.

Code

Output / Result

```

@Model(adaptables = SlingHttpServletRequest.class)
public class PageModel {
    // Sling framework objects
    @SlingObject
    private ResourceResolver resourceResolver;
    @SlingObject
    private Resource resource;
    @SlingObject
    private SlingHttpServletRequest request;
    // AEM HTL script bindings
    @ScriptVariable
    private Page currentPage;
    @ScriptVariable
    private PageManager pageManager;
    public String getPageTitle() {
        return currentPage.getTitle();
    }
}

```

adaptables must include Request for
@ScriptVariable

Source: Sling framework

resolver.getUserID() → "admin"

Source: current component node

resource.getPath() →
".../jcr:content/mycomp"

Source: HTTP request

request.getMethod() → "GET"

Source: HTL bindings

currentPage → Page object

currentPage.getPath() →
"/content/mysite/home"

Source: HTL bindings

pageManager → can resolve any path

getPageTitle() → "My Homepage"

@OSGiService (in Sling Models)

Injects an OSGi service into a Sling Model field. Uses the BundleContext from the model's bundle to look up services by interface type. Supports an optional LDAP filter to select specific implementations.

Code

```

@Model(adaptables = Resource.class)
public class SearchModel {
    // Inject by interface type
    @OSGiService
    private QueryBuilder queryBuilder;
    // For creating other Sling Models
    @OSGiService
    private SlingModelFactory modelFactory;
    // Optional – null if not registered
    @OSGiService
    @Optional
    private BetaFeatureService betaService;
    // LDAP filter – select specific impl
    @OSGiService(filter = "(env=prod)")
    private MyApiService prodApi;
}

```

Output / Result

OSGi registry has QueryBuilder
registered

queryBuilder → QueryBuilder impl

Use to programmatically create models

modelFactory → SlingModelFactory

BetaFeatureService NOT registered

betaService → null (no exception)

Two impls: MyApiServiceProd(env=prod)

MyApiServiceDev(env=dev)

prodApi → MyApiServiceProdImpl

■ **@OSGiService vs @Reference:** @OSGiService is for @Model classes (per-request). @Reference is for @Component classes (bundle lifetime). For multiple services or dynamic binding, you must use @Reference in an @Component class.

@Self and @Via

@Self injects the adaptable itself (the Resource or Request the model was created from), or adapts it to another model type. **@Via** is a modifier that shifts the injection source for a specific field — to a child node, supertype, or forced resource type.

Code

```
// @Self – inject the current adaptable
@Self
private Resource resource;

// @Self – adapt same resource to LinkModel
@Self
private LinkModel linkModel;

// @Via("childName") – read property
// from child node "link", not current node
@ValueMapValue
@Via("link")
private String href;

// @Via ResourceSuperType – read from
// sling:resourceType parent overlay
@ValueMapValue
@Via(type = ResourceSuperType.class)
private String textColor;

// @Via ForcedResourceType – overlay
@ValueMapValue
@Via(type = ForcedResourceType.class,
value = "myapp/base/text")
private String baseText;
```

Output / Result

Current resource node

resource → Resource(.../mycomp)

Sling adapts same resource to LinkModel

linkModel → fully injected LinkModel

JCR: .../mycomp/link/href = "https://..."

href → "https://example.com"

Reads from parent resource type node

textColor → "#333333"

Reads from myapp/base/text overlay

baseText → "Inherited text"

@Exporter — JSON / .model.json endpoint

Enables the Sling Model Exporter framework to serialize a model to JSON. Required for SPA (React/Angular) AEM architectures. Works with Jackson annotations (@JsonProperty, @JsonIgnore) to control the JSON output.

Code

Output / Result

```
@Model(  
    adaptables = Resource.class,  
    adapters = MyTextModel.class,  
    resourceType = "myapp/components/text"  
)  
@Exporter(  
    name = SLING_MODEL_EXPORTER_NAME,  
    extensions = SLING_MODEL_EXTENSION  
)  
public class MyTextModel  
    implements ComponentExporter {  
    @ValueMapValue  
    private String title;  
    @ValueMapValue  
    private String text;  
    @ValueMapValue  
    @JsonIgnore  
    private String _internalId; // excluded!  
    @JsonProperty("heading")  
    public String getTitle() { return title; }  
    public String getText() { return text; }  
    @Override  
    public String getExportedType() {  
        return "myapp/components/text";  
    }  
}
```

Enables `.model.json` endpoint

JCR: title = "Welcome"

JCR: text = "<p>Hello</p>"

JCR: _internalId = "xyz"

NOT in JSON output (@JsonIgnore)

Renamed: "title" key → "heading" in JSON

GET `.../text.model.json` →

```
{  
  "heading": "Welcome",  
  "text": "<p>Hello</p>",  
  ":type": "myapp/components/text"  
}
```

2. OSGi Declarative Services Annotations

OSGi DS annotations govern the bundle lifecycle — service registration, dependency injection between services, configuration management, and lifecycle hooks. These live in `org.osgi.service.component.annotations`.

@Component

Declares a class as an OSGi Declarative Services component. The build tools generate an XML service descriptor automatically. Every OSGi service, event handler, and scheduler starts with `@Component`.

Code

```
@Component(
  service = MyService.class,
  immediate = true,
  property = {
    "service.ranking:Integer=200",
    "label=My Service"
  }
)
public class MyServiceImpl implements MyService {
  @Activate
  protected void activate(ComponentContext ctx) {
    // runs when bundle starts
  }

  @Deactivate
  protected void deactivate() {
    // runs when bundle stops
  }
}
```

Output / Result

Registers as MyService in OSGi registry

immediate=true: start on bundle start

property: service metadata

:Integer → typed as Integer, not String

Needed for service.ranking to work!

Must implement service= interface(s)

@Reference fields bound FIRST

THEN @Activate fires

Safe to use @Reference services here

Runs on bundle stop/unregistration

Clean up connections, threads, caches

Attribute	Purpose	Example
service	Interface(s) to register as	service = MyService.class
immediate	true = activate on bundle start (not lazy)	immediate = true
property	OSGi service properties for registry	"service.ranking:Integer=100"
name	Unique component name (default = FQN)	name = "com.example.MyComp"

@Reference

The primary OSGi wiring annotation. Injects another OSGi service as a dependency. Supports all cardinalities (optional, mandatory, multiple), dynamic binding, and LDAP filters. Always bound before `@Activate` fires.

Code

Output / Result

```
// MANDATORY single (default) – throws if missing
@Reference
private ResourceResolverFactory rrFactory;
// OPTIONAL single – null if not registered
@Reference(cardinality =
ReferenceCardinality.OPTIONAL)
private MyOptionalService optService;
// MULTIPLE – all impls, sorted by ranking
@Reference(
cardinality = ReferenceCardinality.MULTIPLE,
policy = ReferencePolicy.DYNAMIC
)
private volatile List<ContentHandler> handlers;
// FILTERED – LDAP filter targets specific impl
@Reference(target = "(region=APAC)")
private RegionService apacService;
```

OSGi: ResourceResolverFactory
registered

rrFactory → ResourceResolverFactory

MyOptionalService NOT registered

optService → null (no exception
thrown)

OSGi: HandlerA(100), HandlerB(200),
HandlerC(50)

handlers → [HandlerB, HandlerA,
HandlerC]

APACRegionServiceImpl has
(region=APAC)

apacService → APACRegionServiceImpl

■ **volatile required with DYNAMIC:** When using ReferencePolicy.DYNAMIC with List references, the field **MUST** be declared volatile — otherwise the JVM may cache the old list reference and not see updates from services binding/unbinding at runtime.

@Activate, @Modified, @Deactivate

OSGi component lifecycle hooks. **@Activate** fires on startup or first config apply. **@Modified** fires on config change (without restart). **@Deactivate** fires on bundle stop.

Code

```
@Component(service = ApiService.class)
@Designate(ocd = ApiConfig.class)
public class ApiServiceImpl implements ApiService
{
    private String apiUrl;
    private int timeout;
    // Both @Activate and @Modified – same method
    // handles startup AND hot config reload
    @Activate @Modified
    protected void configure(ApiConfig cfg) {
        apiUrl = cfg.api_url();
        timeout = cfg.timeout();
    }
    @Deactivate
    protected void deactivate() {
        // close connections, cancel tasks
        apiUrl = null;
    }
}
```

Output / Result

On bundle start:

apiUrl → "https://api.example.com"

timeout → 5000

Config changes in Felix console:

configure() re-runs automatically

No bundle restart needed!

@ObjectClassDefinition + @AttributeDefinition

Defines an OSGi configuration schema as a Java @interface. Generates the Felix Web Console configuration form automatically. Each method maps to one config property with type, default, and display name.

Code

```
@ObjectClassDefinition(
    name = "API Service Config",
    description = "Settings for ApiService"
)
public @interface ApiConfig {
    @AttributeDefinition(name = "API URL")
    String api_url() default
    "https://api.example.com";

    @AttributeDefinition(name = "Timeout ms")
    int timeout() default 5000;

    @AttributeDefinition(name = "Retry count")
    int retry_count() default 1;

    @AttributeDefinition(name = "Enabled")
    boolean enabled() default true;

    @AttributeDefinition(name = "Allowed paths")
    String[] allowed_paths()
    default {"/content", "/apps"};
}
```

Output / Result

Felix console shows this as form title

cfg.json key: api_url
(underscore→dot)

cfg.api_url() →
"https://api.example.com"

Type: Integer field in Felix console

cfg.timeout() → 5000

Felix renders as number input

cfg.retry_count() → 1

Felix renders as checkbox

cfg.enabled() → true

Felix renders as multi-value list

cfg.allowed_paths() →
["/content", "/apps"]

★ **cfg.json naming:** method name underscores become dots in the .cfg.json key. Example: api_url() method → "api.url" in the .cfg.json file. Always pair @ObjectClassDefinition with @Designate on the @Component class.

@Designate

Links a @Component class to its @ObjectClassDefinition config interface. With **factory=true**, enables multiple independent instances of the same component, each with its own configuration.

Code

Output / Result

```
// Single-instance config (default)
@Component(service = MyService.class)
@Designate(ocd = MyConfig.class)
public class MyServiceImpl implements MyService {
    @Activate
    protected void activate(MyConfig cfg) { }
}

// Factory config – multiple instances
@Component(service = RegionService.class)
@Designate(ocd = RegionConfig.class, factory = true)
public class RegionServiceImpl
implements RegionService {
    @Activate
    protected void activate(RegionConfig cfg) { }
}

// Factory .cfg.json files:
// com.example.RegionServiceImpl-APAC.cfg.json
// com.example.RegionServiceImpl-EMEA.cfg.json
// → creates 2 separate service instances
```

One config per component PID
Pairs @OCD with this component

cfg populated from .cfg.json

factory=true → multiple instances
Each instance has its own config

Each instance activated separately

Instance 1: region=APAC config
Instance 2: region=EMEA config
2 RegionService beans registered!

EventHandler Pattern

Registers a component as an OSGi EventHandler to react to JCR/Sling events. Always use a path filter to avoid firing on every repository change.

Code

```
@Component(
    service = EventHandler.class,
    property = {
        EventConstants.EVENT_TOPIC + "="
        + SlingConstants.TOPIC_RESOURCE_CHANGED,
        EventConstants.EVENT_FILTER
        + "=(path=/content/mysite/*)"
    }
)
public class CacheInvalidator
implements EventHandler {
    @Reference
    private CacheService cacheService;

    @Override
    public void handleEvent(Event event) {
        String path = (String)
        event.getProperty("path");
        if (path != null) {
            cacheService.invalidate(path);
        }
    }
}
```

Output / Result

Must register as EventHandler service

Topic = what type of event to receive
TOPIC_RESOURCE_CHANGED = any JCR change

Filter = restrict by path (critical!)

Without filter: fires on ALL JCR changes

cacheService injected before @Activate

Author edits /content/mysite/en/home

event.getProperty("path")

→ "/content/mysite/en/home"

cacheService.invalidate() called!

Scheduler Pattern (Cron Jobs)

Registers a scheduled job using the Sling Scheduler. The component implements Runnable and declares scheduler.expression as a cron string. Always use try-with-resources for ResourceResolver — request lifecycle never closes it for you.

Code

```
@Component(service = Runnable.class,
property = {
    SchedulerConstants.SCHEDULER_EXPRESSION
+ "=0 0 2 * * ?", // 2am daily
    SchedulerConstants.SCHEDULER_CONCURRENT
+ ":Boolean=false" // no overlap
}
)
public class DailyCleanupJob implements Runnable {
    @Reference
    private ResourceResolverFactory factory;

    @Override
    public void run() {
        try (ResourceResolver rr =
            factory.getServiceResourceResolver(null)) {
            Resource tmp = rr
                .getResource("/var/myapp/tmp");
            // delete stale temp nodes
        } catch (Exception e) {
            // log error
        }
    }
}
```

Output / Result

Must register as Runnable service

Cron expression:

"0 0 2 * * ?" = 02:00:00 every day

concurrent=false:

if prev run active → skip new run

factory injected, ready for run()

try-with-resources: auto-closes rr

CRITICAL: never leak ResourceResolver!

Safely traverses /var/myapp/tmp

3. Key Differences — When to Use Which

@Inject vs @ValueMapValue

■ **Rule:** Use `@ValueMapValue` for all JCR property reads. `@Inject` is a generic fallback — it tries ALL injectors in sequence (slower + ambiguous).

Aspect	@Inject	@ValueMapValue
Source	Any — tries ValueMap, ChildResource, OSGi, ScriptVar, RequestAttr in order	JCR ValueMap only (direct)
Speed	Slower — checks all injectors even after match	Faster — skips to ValueMap directly
Clarity	Ambiguous — reader cannot tell where value comes from	Explicit — always JCR property
Type coercion	Yes (via matched injector)	Yes (built-in: String→int/bool/List)
OSGi service	Yes (via OSGi injector)	No — use @OSGiService
currentPage	Yes (via ScriptVariable injector)	No — use @ScriptVariable
Rename key	@Named("jcr:title") modifier	name="jcr:title" attribute
Default value	@Default modifier	@Default modifier
Use when	Legacy code or unknown source	ALWAYS for JCR properties (new code)

@Optional vs @Required vs @Default

■ **Rule:** `@Default` is safer than `@Optional` for user-facing strings — it prevents null and eliminates null-checks. `@Required` is fail-fast — use for fields that MUST exist (IDs, SKUs).

Aspect	@Optional	@Required	@Default
Missing value	null (or 0/false for primitives)	ModelClassException thrown — model fails	Returns declared fallback value
Model creates?	Yes — always	No — throws on missing value	Yes — fallback used
Null-check needed?	Yes — always	N/A (exception before null)	No — field is never null
Best for	Truly optional UI elements	Required IDs, SKUs, mandatory paths	All user-facing text, labels, URLs
Primitive fields	0 / false (Java default)	Exception if property missing	@Default(intValues=N) sets specific value

@OSGiService (in Models) vs @Reference (in Components)

■ **Rule:** @OSGiService is for @Model classes (per-request). @Reference is for @Component classes (bundle lifetime). Only @Reference supports MULTIPLE cardinality and DYNAMIC binding.

Aspect	@OSGiService (in @Model)	@Reference (in @Component)
Used in	@Model classes only	@Component (OSGi DS) classes only
Cardinality	OPTIONAL and MANDATORY only	OPTIONAL, MANDATORY, MULTIPLE, AT_LEAST_ONE
Dynamic binding	No	Yes — ReferencePolicy.DYNAMIC
Multiple services	No	Yes — List with MULTIPLE
LDAP filter	filter="(prop=val)"	target="(prop=val)"
Lifecycle	Per-request (short-lived)	Bundle lifetime (long-lived)
Bound before	@PostConstruct fires	@Activate fires
volatile needed?	No	Yes — for DYNAMIC multi-references

@ChildResource vs @Self vs @Via

Aspect	@ChildResource	@Self	@Via
Direction	DOWN — navigates to child node	CURRENT — the adaptable itself	LATERAL — different source overlay
Injects as	Resource, List, List	Resource, Request, or other @Model	Modifier — changes another annotation's source
Standalone?	Yes	Yes	No — always modifies @ValueMapValue etc.
List support	Yes — auto-adapts each child	No	No
Colon in name	name="cq:responsive"	N/A	N/A
Use for	Items, slides, nested models	Delegation pattern, raw resource	Property from child, supertype inheritance

Sling Model vs OSGi Lifecycle

Aspect	Sling Model (@Model)	OSGi Component (@Component)
Created when	resource.adaptTo(MyModel.class) is called	Bundle starts (or first reference if not immediate)

Aspect	Sling Model (@Model)	OSGi Component (@Component)
Lifetime	Per-request or per-adaptation (short)	Bundle lifetime (long) — shared singleton
Injection hook	@PostConstruct (after all @ValueMapValue etc.)	@Activate (after all @Reference fields bound)
Teardown hook	None (GC handles it)	@Deactivate (on bundle stop)
Config source	JCR properties (@ValueMapValue)	OSGi .cfg.json (@ObjectClassDefinition)
Service injection	@OSGiService (limited)	@Reference (full cardinality + dynamic)
Thread model	Per-request thread	Can be multi-threaded — use volatile/sync
Good for	Component-level rendering logic	Shared business logic, integrations, jobs

4. Quick Reference — Which Annotation for What

Goal	Annotation	Class type	Notes
Read a JCR property	@ValueMapValue	@Model	Use name= for jcr:title, sling:resourceType etc.
JCR property with fallback	@ValueMapValue + @Default	@Model	@Default(values="x") / intValue / booleanValues
Required JCR property	@ValueMapValue + @Required	@Model	Throws ModelClassException if property missing
Read child node	@ChildResource	@Model	name= for nodes with colons; auto-adapts List
Get current resource	@SlingObject Resource	@Model	Works with both Resource and Request adaptables
Get current page	@ScriptVariable Page	@Model	Needs SlingHttpServletRequest adaptable
Get ResourceResolver	@SlingObject ResourceResolver	@Model	Use for JCR traversal in @PostConstruct
Use an OSGi service	@OSGiService	@Model	filter= for specific impl; @Optional for nullable
Init after injection	@PostConstruct	@Model	Only safe place to use injected values
Inject current adaptable	@Self	@Model	Also adapts same resource to another @Model
Read from child prop	@ValueMapValue + @Via("child")	@Model	@Via shifts the source, not the node injected
Export model as JSON	@Exporter + @JsonProperty	@Model	Requires ComponentExporter + resourceType
Register a service	@Component(service=X.class)	@Component	Must implement the interface listed in service=
Wire service→service	@Reference	@Component	Full cardinality, dynamic, filtered support
Read OSGi config	@Designate + @OCD + @Activate	@Component	@Modified for hot-reload without restart
Multiple service impls	@Reference(cardinality=MULTIPLE)	@Component	volatile List + DYNAMIC policy

Goal	Annotation	Class type	Notes
React to JCR changes	EventHandler + service=EventHandler.class	@Component	ALWAYS add EVENT_FILTER to limit scope
Schedule a job	Runnable + scheduler.expression	@Component	Use try-with-resources for ResourceResolver
Prioritize service	"service.ranking:Integer=N" property	@Component	Higher N = higher priority; use :Integer type!
Multiple configs	@Designate(factory=true)	@Component	Creates ~instanceName config nodes

This guide covers Adobe Experience Manager (AEM) with Sling Models API and OSGi Declarative Services. For the latest API docs visit: sling.apache.org/documentation/bundles/models.html and osgi.org/specification/osgi.cmpn